Student: Todoruț Aurel - Dan

Group: 30434

Program for testing the performance of a PC

Technical University of Cluj-Napoca

Contents

1 Introduction **3**

1.1 Project Proposal . . . 3

1.1.1 Functional requirements . . . 3

1.1.2 Non Functional requirements . . . 4

1.2 Project Timeline . . . 5

2 Bibliographic study **6**

3 Analysis **7**

3.1 System Information Retrieval . . . 7

3.2 CPU Benchmark Design . . . 7

3.2.1 Pi Calculation via numerical integration . . . 7

3.2.2 Matrix multiplication . . . 8

3.2.3 Integer hashing operations . . . 8

3.2.4 Floating Point Trigonometry . . . 8

3.3 Memory benchmark design . . . 9

3.3.1 Cache hierarchy testing . . . 9

3.3.2 Random Access Latency . . . 10

3.4 Storage benchmark design . . . 11

3.4.1 Direct I/O implementation . . . 11

3.4.2 Sequential Performance Tests . . . 12

3.4.3 Random I/O Performance (IOPS) . . . 13

3.5 GPU Benchmark Design . . . 13

3.5.1 Mandelbrot Set Computation (OpenCL) . . . 14

3.6 Scoring System Design . . . 15

3.6.1 Reference Baseline . . . 15

3.6.2 Score Calculation . . . 15

3.6.3 Multi-Iteration Averaging . . . 16

4 Design **16**

4.1 System Architecture . . . 16

4.2 Backend Component Design . . . 17

4.2.1 Module Structure 17
4.3 Frontend Component Design 18
4.3.1 Class Hierarchy 18
4.3.2 User Interface Layout 18

5 Implementation **18**
5.1 C++ Backend Implementation 18
5.1.1 Build System 18
5.2 Python Frontend Implementation 19
5.2.1 Subprocess Communication 19
5.2.2 Scoring Engine 19
5.2.3 User Interface Components 19
5.2.4 Threading Architecture 20
5.2.5 Application Packaging 20

6 Testing and Validation Experiments **21**
6.1 Experimental Setup 21
6.2 Baseline validation (System A) 23
6.3 Comparative performance 25
6.3.1 Low End Performance (System B) 25
6.3.2 High end Performance (System C) 26
6.4 Validation 27

7 Conclusion **28**

8 References **28**

1 Introduction

1.1 Project Proposal

The goal of this project is to develop a desktop app for benchmarking the performance of personal computer components, including the CPU, RAM, GPU, and SSD/HDD. The application provides users with performance metrics through a user friendly graphical interface and a scoring system that compares results against a reference baseline. It offers a solution for general users to quickly test their hardware's capabilities without requiring technical knowledge.

1.1.1 Functional requirements

System information display:

The application detects and displays information about the system's hardware, including CPU model, core count, clock speed, L3 cache size, RAM configuration (manufacturer, capacity, speed, part numbers), GPU model and VRAM, and storage drive models and capacities.

CPU Benchmark:

The CPU test which includes multiple workloads measuring different aspects of processor performance:

- Pi calculation via numerical integration (single and multi-threaded)
- Matrix multiplication (single and multi-threaded)
- Integer hashing operations
- Floating point trigonometry and roots

Memory (RAM) benchmark:

The memory benchmark tests multiple levels of the cache hierarchy:

- L1 Cache bandwidth
- L2 Cache bandwidth
- L3 Cache bandwidth

- Main RAM bandwidth
- Latency

Storage (SSD/HDD) Benchmark

The storage benchmark measures performance using direct I/O:

- Sequential read/write IOPS
- Random read/write IOPS

GPU Benchmark

The GPU test uses OPENCL:

- Mandelbrot set computation

Scoring system

A scoring engine that calculates normalized performance scores:

- Component scores (CPU, RAM, Storage, GPU) on a 1000 point baseline
- Geometric mean aggregation for fair comparison
- Multiple iterations with averaged results
- System score calculated from geometric mean
- Score charts for each test

1.1.2 Non Functional requirements

Performance: The user interface remains responsive during tests through multi threaded execution.

Accuracy: Results are reliable with multiple iterations.

Repeatability: Comparable results across runs.

Portability: Designed for Windows.

1.2 Project Timeline

Week 2	**Week 4**	**Week 6**	**Week 8**
Introduction & bibliographic study	Analysis & design	Backend implementation	UI integration
Week 10	**Week 12**	**Week 14**	
Testing and validation experiments	Conclusion	Presentation	

2 Bibliographic study

The development of an effective PC performance benchmark requires understanding of computer architecture and performance measurement techniques. In [1], the fundamental concepts of CPU performance are explained, including instruction level parallelism, pipelining, and superscalar execution. These concepts informed the design of CPU benchmarks that test not just raw clock speed, but the processor's ability to efficiently handle instruction flows.

Memory performance is examined in [2], which details the performance gap between CPU cache levels and main memory. This source says that accessing main RAM can be hundreds of times slower than accessing L1 cache, necessitating careful benchmark design to measure each level of the memory hierarchy independently.

The distinctions between storage technologies seen in [3], explaining how HDDs are limited by mechanical seek time while SSDs excel at random access patterns. This understanding shaped the choice to measure both sequential IOPS and random IOPS as distinct performance metrics.

For GPU benchmarking, [4] describes the SIMT (Single Instruction, Multiple Threads) execution model that enables massive parallelism. This guided the selection of the Mandelbrot set as an ideal benchmark workload, as each pixel calculation is completely independent and can be computed in parallel across thousands of GPU cores.

[5] provides the foundation for accurate timing measurements through the C++ chrono library and std::chrono::high_resolution_clock, ensuring that all performance measurements are both precise and repeatable.

3 Analysis

3.1 System Information Retrieval

The application retrieves hardware information using the Windows Management Instrumentation (WMI) framework. Key WMI classes queried include:

- **Win32_Processor:** CPU name, core count, logical processor count, max clock speed, L3 cache size
- **Win32_PhysicalMemory:** RAM stick manufacturer, capacity, speed, part number
- **Win32_VideoController:** GPU name, VRAM, driver version
- **Win32_DiskDrive:** Storage device model and capacity

3.2 CPU Benchmark Design

3.2.1 Pi Calculation via numerical integration

The CPU benchmark implements numerical integration to calculate pi using the rectangle method. The integral formula is:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx$$

Algorithm: The implementation divides the interval [0,1] into n rectangles where n is determined by the target digit precision:

Algorithm 1 Pi Calculation (Numerical Integration)

```
step ← 1.0/max_iterations
sum ← 0.0
for i ← 0 max_iterations do
    x ← (i + 0.5) × step
    sum ← sum + 4.0/(1.0+x^2)
end for
```

Single-threaded mode: One thread processes all iterations sequentially, measuring single core performance.

Multi-threaded mode: The iteration space is divided into chunks equal to the number of hardware threads. Each thread computes its chunk independently, and partial sums are combined after all threads complete.

3.2.2 Matrix multiplication

Matrix multiplication serves as the floating point benchmark. For 2048×2048 matrices, the standard algorithm performs:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j]$$

Optimization: The implementation transposes matrix B before multiplication to improve cache locality:

Algorithm 2 Matrix Multiplication

```
Phase 1: Transpose Matrix B
for i ← 0 N do
    for j ← 0 N do
        B_transposed[j][i] ← B[i][j]
    end for
end for
Phase 2: Multiplication
for i ← 0 N do
    for j ← 0 N do
        sum ← 0.0
        for k ← 0 N do
            sum ← sum + A[i][k] × B_transposed[j][k]
        end for
        C[i][j] ← sum
    end for
end for
```

This ensures sequential memory access in the innermost loop, improving cache hit rates. The multi threaded version distributes rows across available threads.

3.2.3 Integer hashing operations

The integer benchmark implements a mixing function using bitwise operations:

This workload stresses integer ALUs with operations resistant to compiler optimization.

3.2.4 Floating Point Trigonometry

The float benchmark chains multiple transcendental functions:

This tests the CPU's FPU performance with mathematical operations.

Algorithm 3 Integer hashing load

```
s_1 ← 0x9E3779B9
s_2 ← 0xBF58476D
for i ← 0 50,000,000 do
    s_1 ← s_1 + i
    s_1 ← RotateLeft(s_1, 13)
    s_2 ← s_2 ⊕ s_1
    s_2 ← s_2 × PrimeConstant
    s_1 ← s_1 + s_2
    s_2 ← RotateLeft(s_2, 27)
    s_1 ← s_1 ⊕ (s_1 ≫ 33)
end for
```

Algorithm 4 Floating-Point FPU Stress Test

```
res ← 1.0
for i ← 0 10,000,000 do
    res ← √(res + 1.5)
    res ← sin(res) × cos(res)
    res ← res + tan(res × 0.001)
    res ← |res|
end for
```

3.3 Memory benchmark design

The memory subsystem benchmark is designed to check the performance of the memory hierarchy, ranging from the L1 cache to the slower main RAM. The design focuses on two metrics: bandwidth and latency.

3.3.1 Cache hierarchy testing

The RAM benchmark isolates specific levels of the memory hierarchy by manipulating the **Working set size**. By allocating a buffer that fits entirely within a specific cache level but exceeds the size of the level below it, we force the CPU to access that specific tier of memory. The sizes chosen reflect standard modern x86 architectures:

- **L1 Cache (32KB):** The fastest memory tier. High iteration count is required to get a measurable elapsed time due to the high speed.
- **L2 Cache (256KB):** Larger but slightly slower than L1.
- **L3 Cache (8MB):** Shared across cores. The 8MB size typically exceeds L2.

- **Main RAM (512MB):** This size is chosen to exceed the L3 cache, ensuring that memory requests miss the cache and fetch data directly from the RAM modules.

Write bandwidth design: The write test utilizes a sequential access pattern. This exploits **Spatial Locality**, allowing the CPU's hardware prefetcher to predict upcoming write addresses and burst transfer full cache lines (64 bytes). Compiler barriers (memory clobbers) are used to prevent the compiler from optimizing away the write operations.

Algorithm 5 Memory Write Bandwidth Measurement

```
for k ← 0 iterations do
    for i ← 0 elements do
        Buffer[i] ← i + k
    end for
end for
Bandwidth ← (elements × sizeof(int) × iterations)/Time
```

Read bandwidth design: The read test employs **loop unrolling** (processing 4 elements per iteration). This technique increases the density of memory instructions relative to branch/loop overhead instructions. It also exposes Instruction Level Parallelism (ILP), allowing the CPU pipeline to issue multiple load requests simultaneously before the previous ones complete, saturating the memory bus.

Algorithm 6 Memory Read Bandwidth Test

```
for k ← 0 iterations do
    for i ← 0 elements step 4 do
        sum_0 ← sum_0 + Buffer[i]
        sum_1 ← sum_1 + Buffer[i + 1]
        sum_2 ← sum_2 + Buffer[i + 2]
        sum_3 ← sum_3 + Buffer[i + 3]
    end for
end for
Bandwidth ← BytesTransferred / Time_elapsed
```

Bandwidth is calculated as:

$$\text{Bandwidth (GB/s)} = \frac{\text{bytes transferred} \times \text{iterations}}{\text{elapsed time (s)} \times 10^9}$$

3.3.2 Random Access Latency

Measuring raw memory latency requires defeating two hardware optimizations: **Caching** and **Hardware Prefetching**.

1. **Defeating locality:** The test initializes a linked list (pointer chain) where every node points to a random index in the array. This ensures memory accesses jump unpredictably across the 512MB buffer, causing constant cache misses.

2. **Dependency chain:** Unlike bandwidth tests where multiple requests can be parallelized, this algorithm creates a strict serial dependency. The CPU cannot compute the address of the *next* node until the data for the *current* node has arrived from RAM.

This "Pointer chasing" technique stalls the CPU pipeline entirely for the duration of the memory round trip, allowing us to measure the pure latency of the memory controller and RAM modules.

Algorithm 7 Random Access Latency (Pointer Chasing)

```
Setup: Create linked chain in random order
for i ← 0 elements − 1 do
    Buffer[indices[i]] ← indices[i + 1]
end for
Measurement:
current ← indices[0]
StartTimer()
for k ← 0 10,000,000 do
    current ← Buffer[current]
end for
StopTimer()
```

Average latency per access is computed as:

$$\text{Latency (ns)} = \frac{\text{total time (ns)}}{\text{number of accesses}}$$

3.4 Storage benchmark design

The storage benchmark is designed to measure the physical performance of the disk subsystem, isolating it from Operating System optimizations. Modern Windows environments heavily utilize RAM to cache file operations; without specific countermeasures, a "storage" benchmark would often just measure RAM speed.

3.4.1 Direct I/O implementation

To ensure data is committed to the physical media, the benchmark bypasses the Windows System File Cache. This is achieved using specific Win32 API flags during file creation:

- `FILE_FLAG_NO_BUFFERING`: Disables the internal OS caching of file data. This forces the OS to transfer data directly from the application buffer to the disk.
- `FILE_FLAG_WRITE_THROUGH`: Instructs the disk controller to flush data to the physical storage medium (NAND Flash) immediately, rather than holding it in the drive's volatile DRAM cache. This prevents artificial "burst" speeds during write operations.

```
HANDLE hFile = CreateFileA(path,
    GENERIC_WRITE, 0, NULL, CREATE_ALWAYS,
    FILE_FLAG_NO_BUFFERING | FILE_FLAG_WRITE_THROUGH,
    NULL);
```

3.4.2 Sequential Performance Tests

Sequential tests measure throughput when reading or writing large contiguous blocks (1MB). This mimics heavy file transfers (e.g., video editing, file copying).

Sequential Write The write test fills the buffer with **random data** before execution. This is critical to defeat hardware compression used by modern SSD controllers. If the buffer contained zeros, the controller would compress the data, resulting in artificially high speeds. The `WRITE_THROUGH` flag is used to ensure data hits the flash storage.

Algorithm 8 Sequential Write Test

```
Handle ← OpenDiskFile(NoBuffering|WriteThrough)
Buffer ← AlignedAlloc(1 MB)
FillWithRandomData(Buffer)                                  ▷
Timer.Start
for i ← 0 num_blocks do
    WriteFile(Handle, Buffer, 1 MB)
end for
Timer.Stop
Bandwidth ← (num_blocks × 1MB)/Timer.Elapsed
```

Sequential Read The read test measures how fast data can be streamed from the drive. Unlike the write test, `WRITE_THROUGH` is not required. A "warm-up" read is performed before the timer starts to ensure the file system driver is ready, preventing startup latency from skewing the result.

Algorithm 9 Sequential Read Test

```
Handle ← OpenDiskFile(NoBuffering)
Buffer ← AlignedAlloc(1 MB)
ReadFile(Handle, Buffer)                ▷
SetFilePointer(Handle, 0)               ▷
Timer.Start
for i ← 0 num_blocks do
    ReadFile(Handle, Buffer, 1 MB)
end for
Timer.Stop
```

3.4.3 Random I/O Performance (IOPS)

Random tests perform small (4KB) transfers at non-sequential offsets to measure Input/Output Operations Per Second (IOPS). This mimics OS booting and application loading behavior.

Random Read IOPS This test constantly moves the file pointer to random locations within the file. It challenges the SSD's mapping table or the HDD's mechanical arm.

Algorithm 10 Random Read IOPS (4KB)

```
Handle ← OpenDiskFile(NoBuffering)
Buffer ← AlignedAlloc(4 KB)
Operations ← 0
Timer.Start
while Timer.Elapsed < 2.0 seconds do
    Offset ← Random(0, FileSize)
    SetFilePointer(Handle, Offset)
    ReadFile(Handle, Buffer)
    Operations ← Operations + 1
end while
IOPS ← Operations/Timer.Elapsed
```

Random Write IOPS Similar to the read test, but writes data to random offsets. This test enables `WRITE_THROUGH` again to prevent the drive from caching these small writes in its DRAM, which would yield misleadingly high IOPS figures.

3.5 GPU Benchmark Design

The GPU benchmark evaluates two distinct aspects of graphics hardware: **Compute Performance** (GFLOPS) using OpenCL.

Algorithm 11 Random Write IOPS (4KB)

```
Handle ← OpenDiskFile(NoBuffering|WriteThrough)
Buffer ← AlignedAlloc(4 KB)
FillWithRandomData(Buffer)
Timer.Start
while Timer.Elapsed < 2.0 seconds do
    Offset ← Random(0, FileSize)
    SetFilePointer(Handle, Offset)
    WriteFile(Handle, Buffer)
    Operations ← Operations + 1
end while
```

3.5.1 Mandelbrot Set Computation (OpenCL)

The Compute test utilizes OpenCL to execute a parallel kernel across thousands of GPU cores. The Mandelbrot set generation is a good stress test because:

1. **Parallel:** Each pixel is independent of others.
2. **Compute bound:** It requires heavy floating point arithmetic (multiplication/addition) rather than memory bandwidth.
3. **Branch divergence:** The `while` loop exits at different times for different pixels, testing the GPU scheduler's ability to handle divergence.

Algorithm 12 GPU Mandelbrot Kernel (OpenCL)

```
procedure MandelbrotKernel(output, max_iters)
    gid ← get_global_id(0)
    Calculate normalized coordinates (x, y) from gid
    Map to complex plane (c_re, c_im)
    z_re ← 0, z_im ← 0
    while i < max_iters and (z_re^2 + z_im^2) < 4.0 do
        new_re ← (z_re^2 − z_im^2) + c_re
        new_im ← (2.0 × z_re × z_im) + c_im
        z_re ← new_re, z_im ← new_im
    end while
    output[gid] ← z_re^2 + z_im^2
end procedure
```

GFLOPS Calculation: Each iteration of the complex number calculation involves 7 floating point operations (4 multiplications, 3 additions/subtractions).

$$\text{GFLOPS} = \frac{\text{Total Ops}}{\text{Execution Time (s)} \times 10^9}$$

3.6 Scoring System Design

The scoring engine aggregates performance metrics (Seconds, GB/s, IOPS) into a normalized, score that allows for easy comparison between different systems.

3.6.1 Reference Baseline

A theoretical "Reference System" is defined, representing a standard mid range configuration. If a user system matches these metrics exactly, it achieves a score of 1000.

Component	Metric	Reference Value
CPU	Pi Single Thread	2.60 s
	Pi Multi Thread	0.50 s
	Matrix Single Thread	2.10 s
	Matrix Multi Thread	0.70 s
	Integer Hashing	0.10 s
	Float Math	0.35 s
RAM	L1 Bandwidth	66.0 GB/s
	L2 Bandwidth	60.0 GB/s
	L3 Bandwidth	50.0 GB/s
	Main RAM Bandwidth	21.0 GB/s
	Latency	114.0 ns
Storage	Seq Read	2,500 IOPS
	Seq Write	3,300 IOPS
	Random Read	15,000 IOPS
	Random Write	35,000 IOPS
GPU	Rasterization	430.0 FPS
	Compute	3,900.0 GFLOPS

Table 1: Reference System Baseline Configuration

3.6.2 Score Calculation

Performance ratios (r_i) are calculated depending on whether the metric is "Higher is Better" (Bandwidth, IOPS, FPS) or "Lower is Better" (Time, Latency).

- **Higher is Better:** $r_i = \frac{\text{Measured Value}}{\text{Reference Value}}$
- **Lower is Better:** $r_i = \frac{\text{Reference Value}}{\text{Measured Value}}$

To prevent a single outlier metric (e.g., an incredibly fast L1 cache) from artificially inflating the score of a slow system, the **Geometric Mean** is used to aggregate the ratios. This provides a balanced view of the component's performance.

$$\text{Component Score} = 1000 \times \left(\prod_{i=1}^{n} r_i \right)^{1/n}$$

3.6.3 Multi-Iteration Averaging

To account for Operating System jitter, background tasks, and thermal throttling, each test suite is executed for $N = 10$ iterations. The component score displayed to the user is the arithmetic mean of the scores from all iterations:

$$\text{Final Component Score} = \frac{1}{10} \sum_{j=1}^{10} \text{Score}_j$$

The overall **System Score** is calculated as the geometric mean of the four primary component scores:

$$\text{System Score} = (\text{Score}_{CPU} \times \text{Score}_{RAM} \times \text{Score}_{Storage} \times \text{Score}_{GPU})^{1/4}$$

4 Design

4.1 System Architecture

The application has a modular two tier architecture separating concerns between performance measurement and user presentation:

Backend (C++): Handles all hardware interaction, benchmarking logic, and WMI queries. Compiled as a standalone executable that communicates via JSON on stdout.

Frontend (Python): Provides the GUI using Tkinter, manages subprocess execution, parses results, implements the scoring engine, and visualizes performance data through custom chart components.

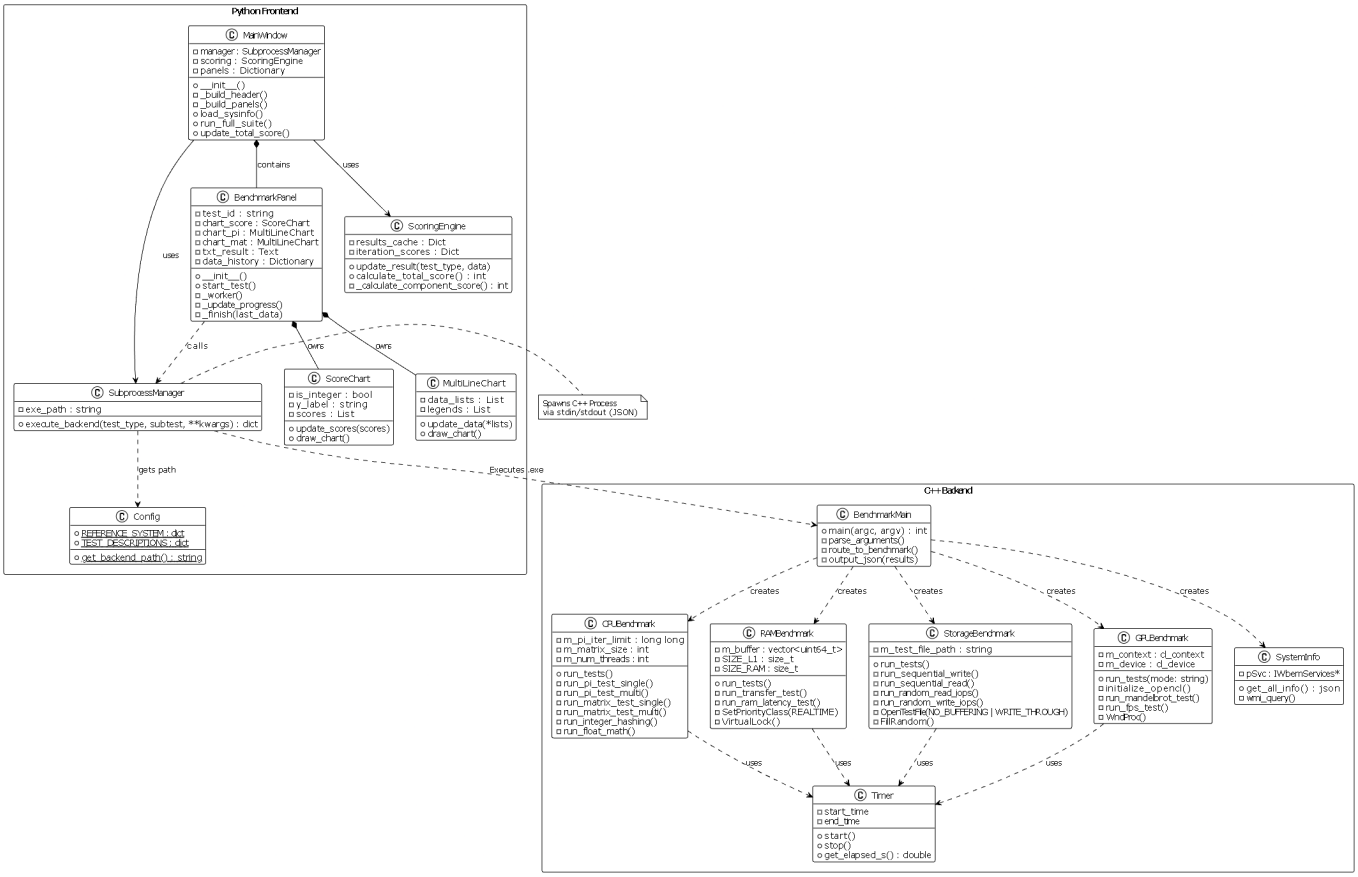


Figure 1: System Architecture Diagram

4.2 Backend Component Design

4.2.1 Module Structure

- **Timer.h:** Wrapper around std::chrono::high_resolution_clock
- **CPUBenchmark:** Implements all CPU tests with thread management
- **RAMBenchmark:** Memory hierarchy testing with cache control
- **StorageBenchmark:** Direct I/O testing with aligned buffers
- **GPUBenchmark:** OpenCL kernel execution and profiling
- **SystemInfo:** WMI integration for hardware detection

4.3 Frontend Component Design

4.3.1 Class Hierarchy

- **MainWindow:** Root Tkinter application with scrollable canvas
- **BenchmarkPanel:** Reusable test panel with progress tracking
- **ScoreChart:** Custom Canvas widget for visualizing score trends
- **SubprocessManager:** Backend communication layer
- **ScoringEngine:** Score calculation and aggregation

4.3.2 User Interface Layout

- **Header Section:** Displays overall system score, component scores, and comparison to baseline
- **System Info Card:** Shows detected hardware in formatted text
- **Test Cards:** Four collapsible panels for CPU, RAM, Storage, GPU tests
- **Each Card Contains:**
  - Test description and "Run Test (5x)" button
  - Real time score progression chart
  - Progress bar showing current iteration
  - Detailed results text area with formatted metrics

5 Implementation

5.1 C++ Backend Implementation

5.1.1 Build System

The project uses CMake as the build system to manage dependencies and compile the C++ backend. The configuration links the OpenCL library required for GPU benchmarking functionality.

5.2 Python Frontend Implementation

5.2.1 Subprocess Communication

The frontend spawns the C++ backend as a subprocess with appropriate command line arguments. Standard output is captured and parsed as JSON. Windows startup flags hide the console window for a cleaner user experience.

5.2.2 Scoring Engine

Reference Baseline: A predefined baseline system with known performance metrics serves as the comparison reference. Each component has specific baseline values (e.g., CPU times, RAM bandwidth, storage IOPS, GPU GFLOPS).

Ratio Calculation: For time based metrics (CPU, cache), the ratio is baseline divided by measured (lower time is better). For throughput metrics (bandwidth, IOPS, GFLOPS), the ratio is measured divided by baseline (higher is better).

Geometric Mean Aggregation: Component scores use geometric mean to combine multiple test results. This prevents one excellent metric from masking poor performance in others. The formula multiplies all ratios and takes the nth root.

Multi Iteration Averaging: Each test runs 5 times, and scores are averaged to reduce noise from background processes, thermal throttling, and operating system scheduling.

5.2.3 User Interface Components

Score Bar Chart Widget: A custom Canvas widget that plots iteration scores as a bar chart. The chart dynamically scales based on minimum and maximum values.

Benchmark Panel: Each test has a collapsible card containing description text, a run button, progress indicator, score chart, and formatted results area. The panel updates dynamically during test execution.

Results Formatting: Raw JSON data is transformed into readable text with proper units, separators, and columns. Test order is predefined to ensure logical presentation (e.g., single thread before multi thread).

Auto Resizing Text Areas: Result displays automatically adjust height based on content length by counting text lines and accounting for line wrapping. This prevents unnecessary scrolling for short results.

5.2.4 Threading Architecture

Benchmark execution runs in background threads to prevent UI freezing. The worker thread communicates with the UI thread using Tkinter's after() method to safely update widgets from the background.

5.2.5 Application Packaging

PyInstaller bundles the Python application, Tkinter libraries, and C++ backend executable into a single file. This eliminates the need for users to install Python or manage dependencies.

6 Testing and Validation Experiments

To validate the accuracy and scoring logic of the application, experiments were done on three distinct hardware configurations. The objective was to confirm that the scoring engine normalizes results against the reference baseline and accurately reflects the performance between different tiers of hardware.

6.1 Experimental Setup

Three systems were selected for validation. The detected hardware configurations for each test are presented below:

1. **System A (Reference):** The personal laptop used to define the baseline metrics ($Score \approx 1000$).

2. **System B (Low-End):** A desktop PC with older generation hardware to verify lower bound scoring ($Score < 1000$).

3. **System C (High-End):** A high performance PC to verify upper bound scaling ($Score > 1000$).

System Hardware

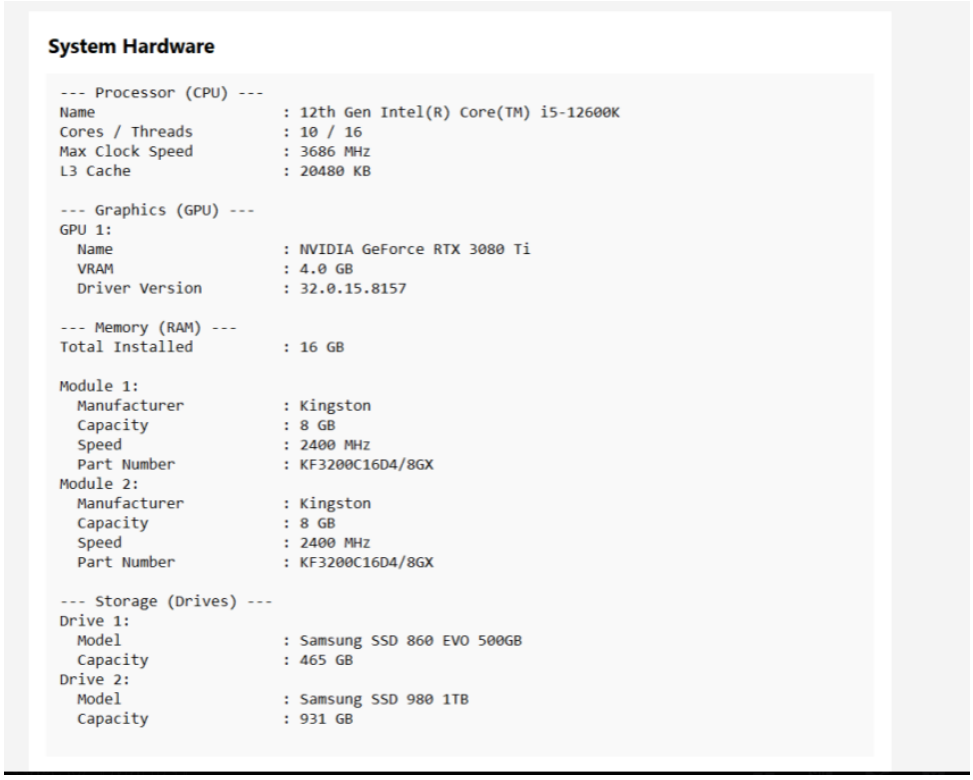
```
--- Processor (CPU) ---
Name                    : AMD Ryzen 5 6600H with Radeon Graphics
Cores / Threads         : 6 / 12
Max Clock Speed         : 3301 MHz
L3 Cache                : 16384 KB

--- Graphics (GPU) ---
GPU 1:
  Name                  : AMD Radeon(TM) Graphics
  VRAM                  : 0.5 GB
  Driver Version        : 31.0.12044.3
GPU 2:
  Name                  : NVIDIA GeForce RTX 3050 Laptop GPU
  VRAM                  : 4.0 GB
  Driver Version        : 32.0.15.7703

--- Memory (RAM) ---
Total Installed         : 8 GB

Module 1:
  Manufacturer          : Samsung
  Capacity              : 8 GB
  Speed                 : 4800 MHz
  Part Number           : M425R1GB4BB0-CQKOL

--- Storage (Drives) ---
Drive 1:
  Model                 : MTFDKBA512TFH-1BC1AABHA
  Capacity              : 476 GB
```

Figure 2: Hardware Specifications - System A (Reference)

System Hardware

```
--- Processor (CPU) ---
Name                    : Intel(R) Core(TM) i5-10400F CPU @ 2.90GHz
Cores / Threads         : 6 / 12
Max Clock Speed         : 2901 MHz
L3 Cache                : 12288 KB

--- Graphics (GPU) ---
GPU 1:
  Name                  : NVIDIA GeForce GTX 1650
  VRAM                  : 4.0 GB
  Driver Version        : 32.0.15.7688

--- Memory (RAM) ---
Total Installed         : 16 GB

Module 1:
  Manufacturer          : A-DATA
  Capacity              : 8 GB
  Speed                 : 2666 MHz
  Part Number           :
Module 2:
  Manufacturer          : A-DATA
  Capacity              : 8 GB
  Speed                 : 2666 MHz
  Part Number           :

--- Storage (Drives) ---
Drive 1:
  Model                 : HP SSD S700 500GB
  Capacity              : 465 GB
```

Figure 3: Hardware Specifications - System B (Low End)



System Hardware

```
--- Processor (CPU) ---
Name                    : 12th Gen Intel(R) Core(TM) i5-12600K
Cores / Threads         : 10 / 16
Max Clock Speed         : 3686 MHz
L3 Cache                : 20480 KB

--- Graphics (GPU) ---
GPU 1:
  Name                  : NVIDIA GeForce RTX 3080 Ti
  VRAM                  : 4.0 GB
  Driver Version        : 32.0.15.8157

--- Memory (RAM) ---
Total Installed         : 16 GB

Module 1:
  Manufacturer          : Kingston
  Capacity              : 8 GB
  Speed                 : 2400 MHz
  Part Number           : KF3200C16D4/8GX
Module 2:
  Manufacturer          : Kingston
  Capacity              : 8 GB
  Speed                 : 2400 MHz
  Part Number           : KF3200C16D4/8GX

--- Storage (Drives) ---
Drive 1:
  Model                 : Samsung SSD 860 EVO 500GB
  Capacity              : 465 GB
Drive 2:
  Model                 : Samsung SSD 980 1TB
  Capacity              : 931 GB
```

Figure 4: Hardware Specifications - System C (High End)

6.2 Baseline validation (System A)

The first experiment involved running the full benchmark suite on System A. Since the reference baseline values in the code were derived from this specific hardware configuration, the expected result is a Total System Score close to 1000.

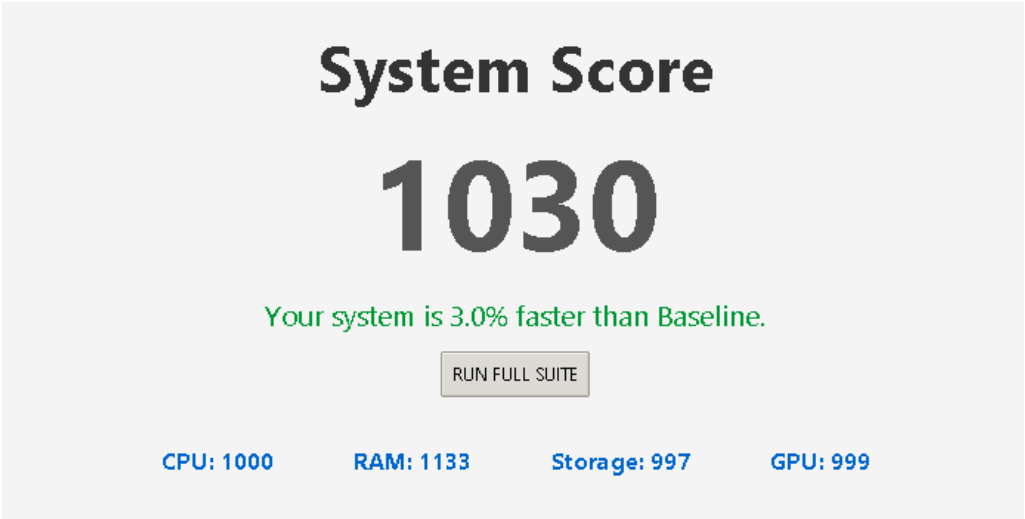


Figure 5: Overall Benchmark Score - System A

Component analysis (System A): Detailed breakdowns of the CPU, Memory, Stor-

age, and GPU performance for the reference system are shown below.

Figure 6: System A: CPU Results

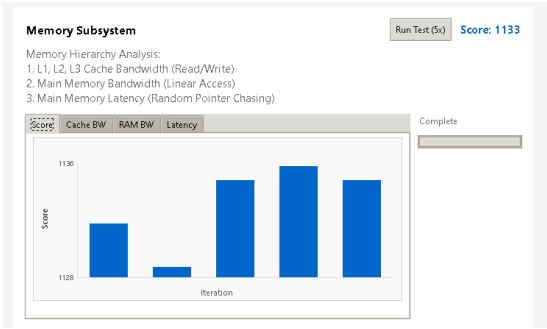


Figure 7: System A: Memory Results

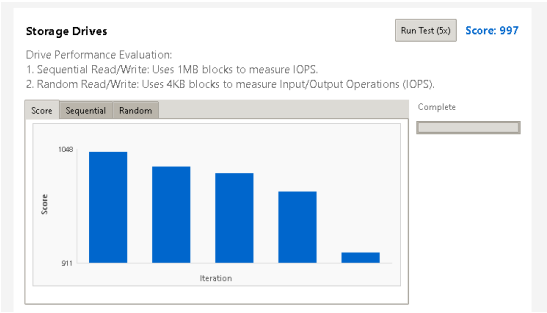


Figure 8: System A: Storage Results

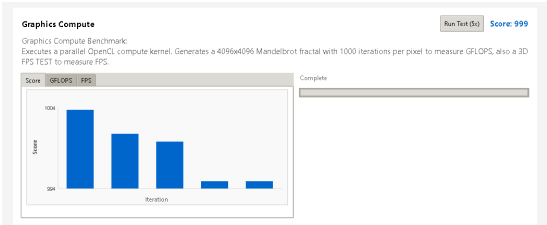


Figure 9: System A: GPU Results

Analysis: As shown in Figure 5, the system achieved a score of approximately **1030**. This slight deviation ($+3\%$) from the theoretical perfect 1000 is expected. This validates the baseline reference.

6.3 Comparative performance

6.3.1 Low End Performance (System B)

System B represents an older configuration. The expectation was lower scores across all categories.

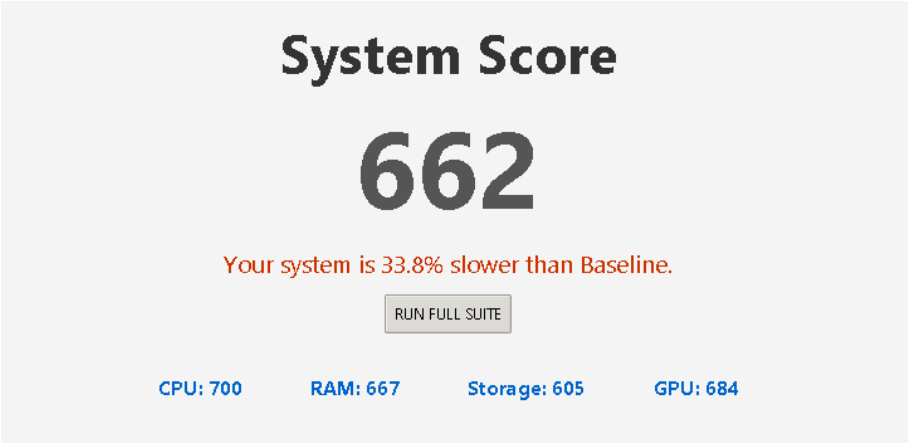


Figure 10: Overall Benchmark Score - System B

Component analysis (System B): The detailed results below highlight the specific bottlenecks of the older hardware.

Analysis: System B achieved a total score of **[662]**.

- The **CPU score** was limited by the older architecture.
- The **Storage score** dropped due to the limitations of the SATA interface.
- The **RAM** score is lower due to the lower Mhz even though 2 RAM sticks are available (DDR4).
- The **GPU** score is expected.

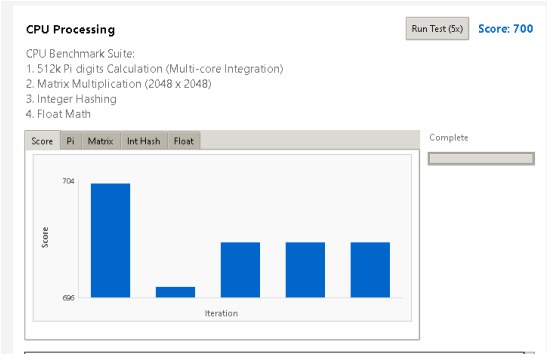


Figure 11: System B: CPU Results

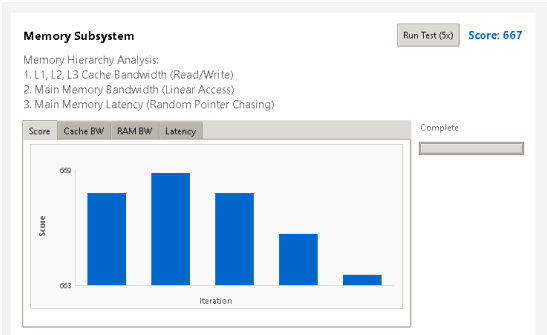


Figure 12: System B: Memory Results

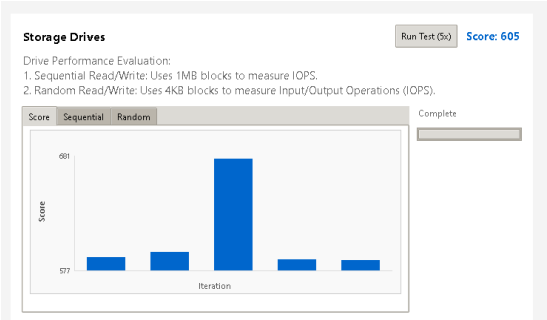


Figure 13: System B: Storage Results

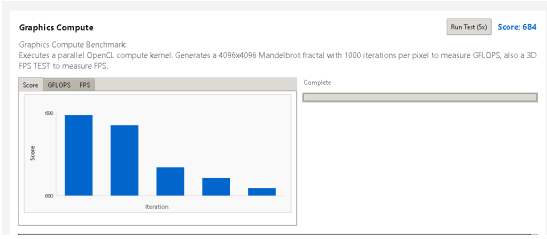


Figure 14: System B: GPU Results

6.3.2 High end Performance (System C)

System C represents a more modern configuration. The expectation was to exceed the baseline score of 1000.

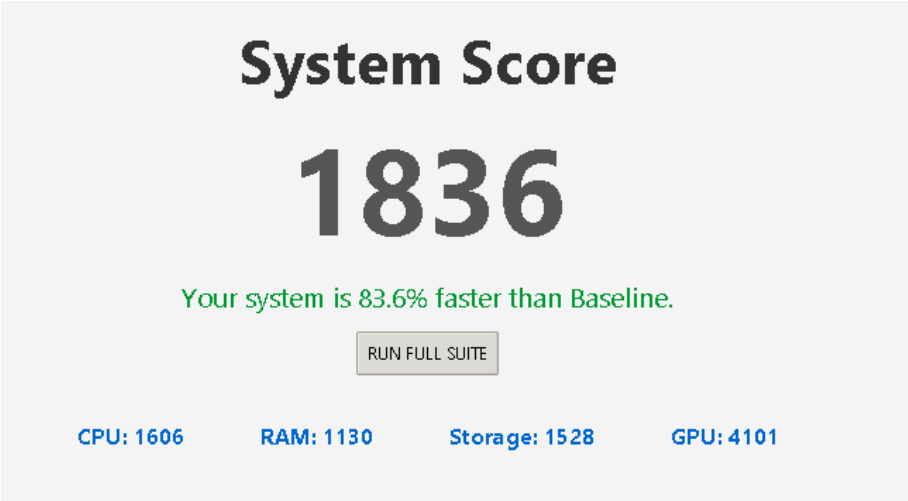


Figure 15: Overall Benchmark Score - System C

Component analysis (System C): The results below demonstrate the scaling capabilities of the benchmark on high end hardware.

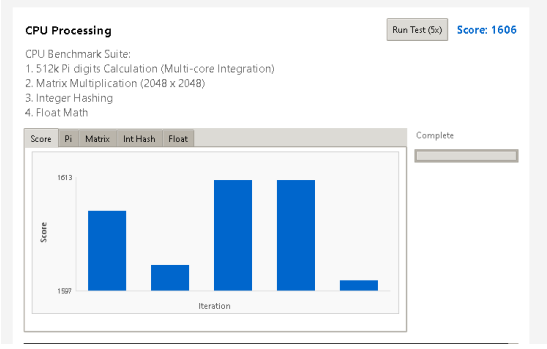


Figure 16: System C: CPU Results

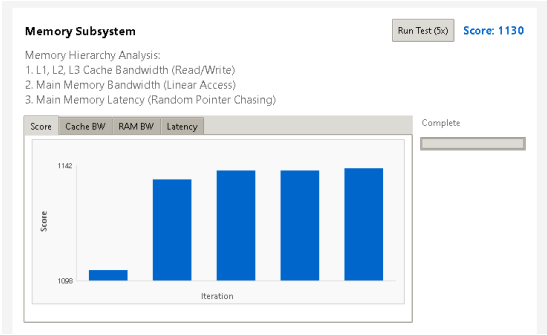


Figure 17: System C: Memory Results

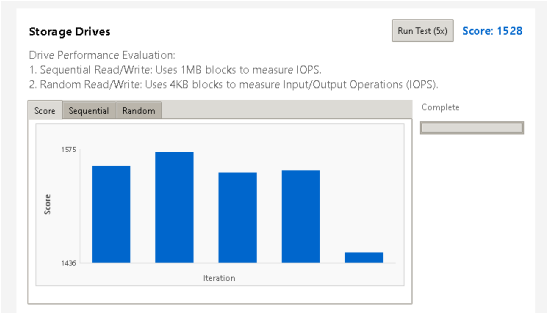


Figure 18: System C: Storage Results

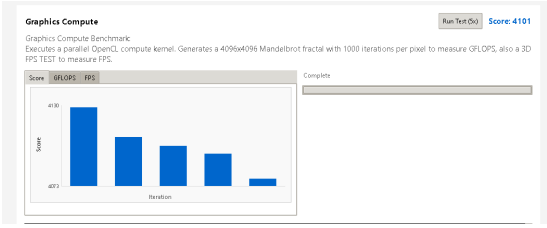


Figure 19: System C: GPU Results

Analysis: System C achieved a total score of **[1836]**.

- **CPU:** The higher core count resulted in a good speedup in the Multi threaded Matrix and Pi tests.
- **GPU:** The newer architecture provided higher GFLOPS and FPS, pushing the component score well above 1000.
- **Memory:** The memory score is the same because 2 RAM sticks are present and the XMP profile is activated (3200 real Mhz).

6.4 Validation

The testing confirmed that the application is reliable.

1. **Normalization:** The reference system scores near 1000, weaker systems score lower, and stronger systems score higher.

2. **Stability:** The application ran on different hardware configurations without issues.

7 Conclusion

The development of the PC Performance Benchmark application met the project's objective: creating a good, user friendly, and accurate tool for hardware performance. By implementing a hybrid architecture, a C++ backend with a Python frontend, the application has both measurement precision and ease of use.

8 References

1. Patterson, D. A., & Hennessy, J. L. (2013). Computer Organization and Design: The Hardware/Software Interface. Morgan Kaufmann.

2. Drepper, U. (2007). What Every Programmer Should Know About Memory. https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

3. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.

4. NVIDIA Corporation. (2023). CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

5. C++ Standards Committee. (2020). ISO/IEC 14882:2020 Programming languages — C++. https://en.cppreference.com/w/cpp/chrono/high_resolution_clock